

Project: Serverless Web Application with AWS Lambda and API Gateway

Services: AWS Lambda, API Gateway, Amazon S3, DynamoDB, Amazon Cognito

Description: Create a fully serverless web application where users can register and log in using Amazon Cognito, upload files to Amazon S3, and store file metadata in DynamoDB. API Gateway manages API requests and securely invokes Lambda functions to process uploads and metadata storage.

Skills: Serverless architecture, API integration, authentication, file storage, database integration.

Definition: The Serverless Web Application is designed to provide a scalable, secure, and fully managed web application architecture using AWS serverless services. Users authenticate using Amazon Cognito, upload files through API Gateway, and AWS Lambda processes the requests by storing uploaded files in Amazon S3 while saving metadata into DynamoDB. This architecture eliminates the need for managing servers and provides automatic scaling, high availability, and cost optimization.

Scope of the Project: The scope of this project is to design and implement a fully serverless web application using AWS managed services. The project focuses on authentication, file upload management, metadata storage, and API integration.

This project covers:

- **User Authentication:** Implements secure user registration and login using Amazon Cognito.
- **File Upload Management:** Allows authenticated users to upload files to Amazon S3.
- **Metadata Storage:** Stores file details such as filename, upload time, and user information in DynamoDB.
- **API Management:** Uses API Gateway to securely expose APIs and route requests to Lambda functions.
- **Fully Serverless Architecture:** No server provisioning or infrastructure management is required.

Methodology:

1. Lambda + API Gateway + S3 + DynamoDB + Cognito – Best Method

- Uses all required AWS services: Lambda, API Gateway, S3, DynamoDB, and Cognito.
- Provides secure authentication and authorization
- Users are authenticated using Cognito User Pools.
- Fully serverless architecture
- No EC2 instances or server management required.
- Automatically scalable
- AWS services scale automatically based on traffic.
- Cost-effective
- Pay only for actual usage.
- Production-ready architecture
- Suitable for enterprise and cloud-native applications.
- Requires API integration and Lambda scripting
- Provides maximum flexibility and scalability.

2. EC2 + Traditional Web Server – Alternative Method

- Uses EC2 instances with Apache/Nginx and backend applications.
- Requires manual server setup and management.
- Authentication handled using application frameworks.
- File uploads stored in S3 or local storage.
- Requires patching, scaling, monitoring, and maintenance.

- Higher operational overhead and infrastructure cost.
- Suitable for legacy applications requiring persistent servers.

AWS Services used in the Project:

1. AWS Lambda

- **Definition:** AWS Lambda is a serverless compute service that allows users to run backend code without managing servers.
- **Purpose:** Processes API requests and handles business logic.
- **Role:**
 - * Receives requests from API Gateway.
 - * Validates authenticated users.
 - * Uploads files to S3.
 - * Stores metadata in DynamoDB.

2. Amazon API Gateway

- **Definition:** Amazon API Gateway is a fully managed service for creating, publishing, and managing APIs.
- **Purpose:** Acts as the frontend API layer for the application.
- **Role:**
 - * Receives HTTP API requests.
 - * Invokes Lambda functions.
 - * Integrates authentication with Cognito.
 - * Manages API security and throttling.

3. Amazon S3 (Simple Storage Service)

- **Definition:** Amazon S3 is an object storage service that provides scalable and durable storage.
- **Purpose:** Stores uploaded files securely.
- **Role:**
 - * Stores user-uploaded files.
 - * Provides scalable cloud storage.
 - * Supports secure access policies.

4. Amazon DynamoDB

- **Definition:** Amazon DynamoDB is a fully managed NoSQL database service.
- **Purpose:** Stores application metadata.
- **Role:**
 - * Stores file upload details.
 - * Maintains user file records.
 - * Supports fast and scalable queries.

5. Amazon Cognito

- **Definition:** Amazon Cognito is a service for user authentication, authorization, and user management.
- **Purpose:** Provides secure login and signup functionality.
- **Role:**

* Authenticates users.

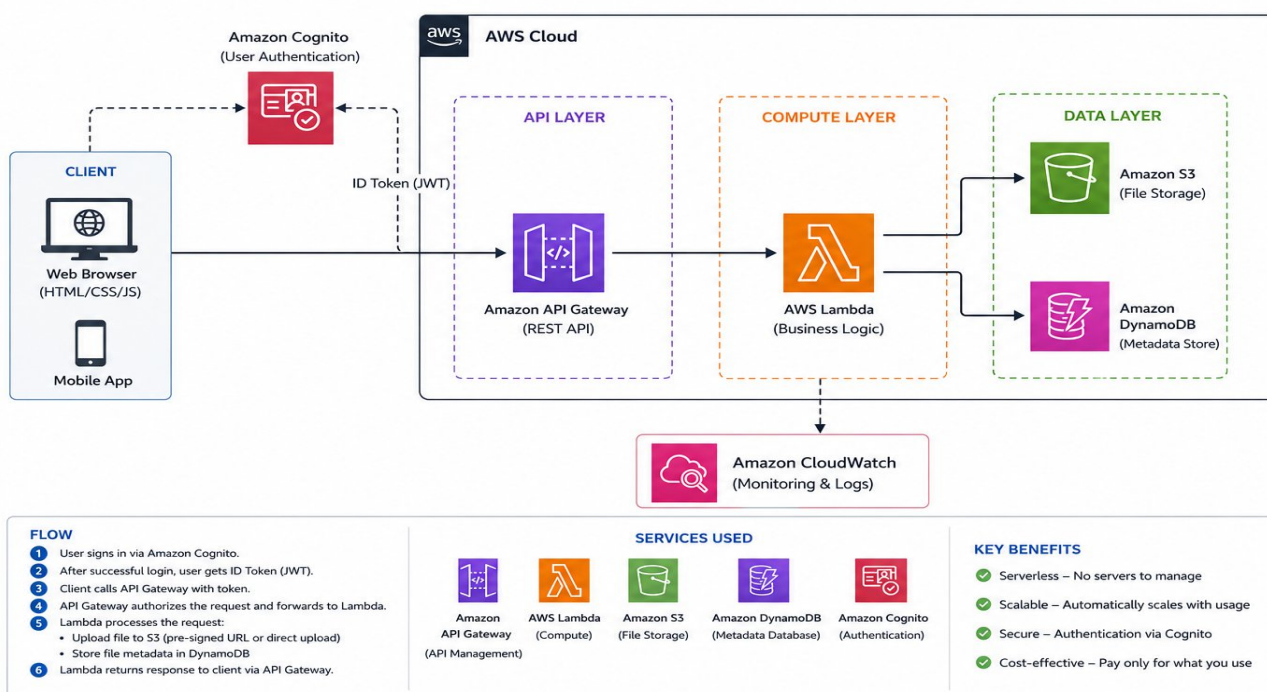
* Generates authentication tokens.

* Secures API access through Cognito Authorizers.

Architecture Diagram:

Serverless Web Application with AWS Lambda and API Gateway

Users can log in (Cognito), upload files (S3), and store metadata (DynamoDB). API Gateway manages API requests.

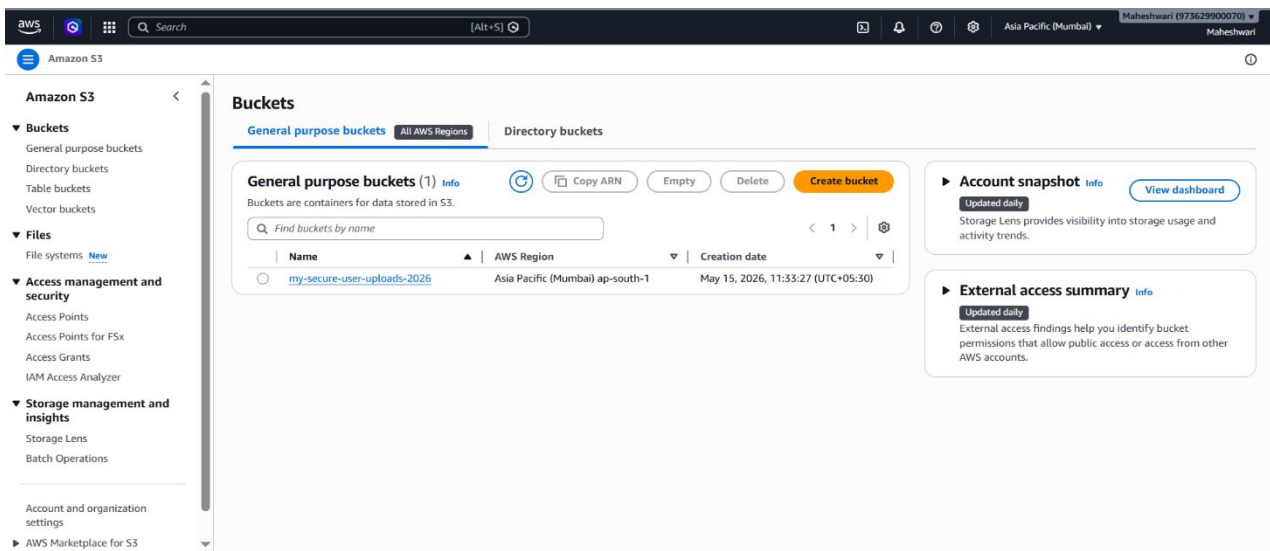


Implementation Steps to Do the Project:

Serverless Web Application

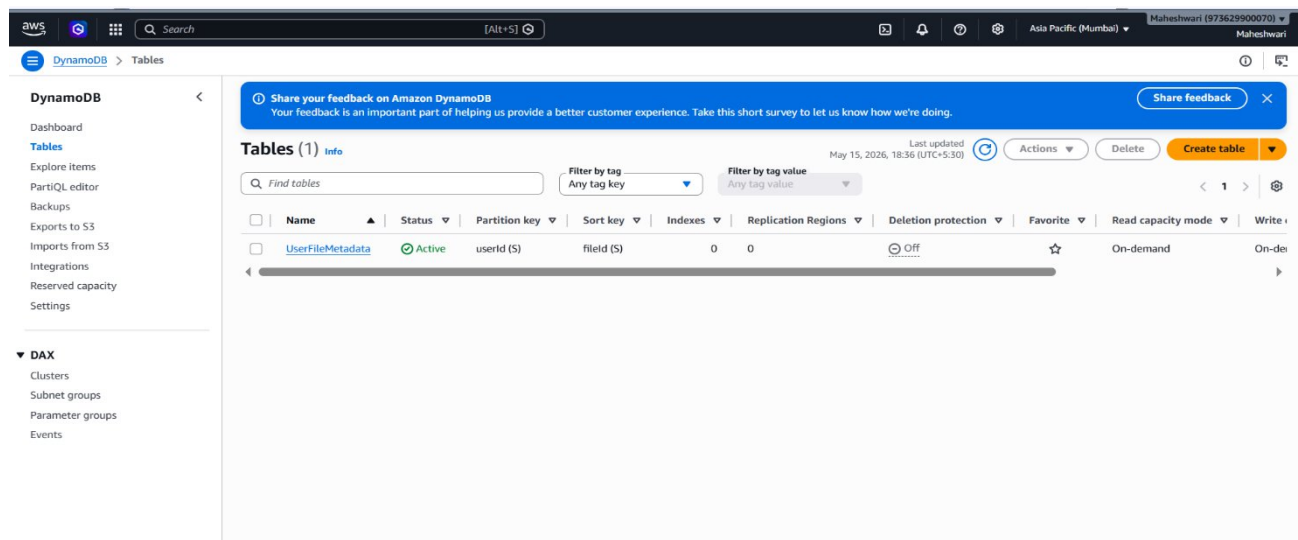
Step 1: Create S3 Bucket

- Go to AWS Console → S3 → Create Bucket.
- Bucket Name: my-secure-user-uploads-2026
- Keep default settings.
- Disable Block Public Access only if required.
- Create Bucket.



Step 2: Create DynamoDB Table

- Go to DynamoDB → Create Table.
- Table Name: UserFileMetadata
- Partition Key: userId
- Keep default settings.
- Create Table.



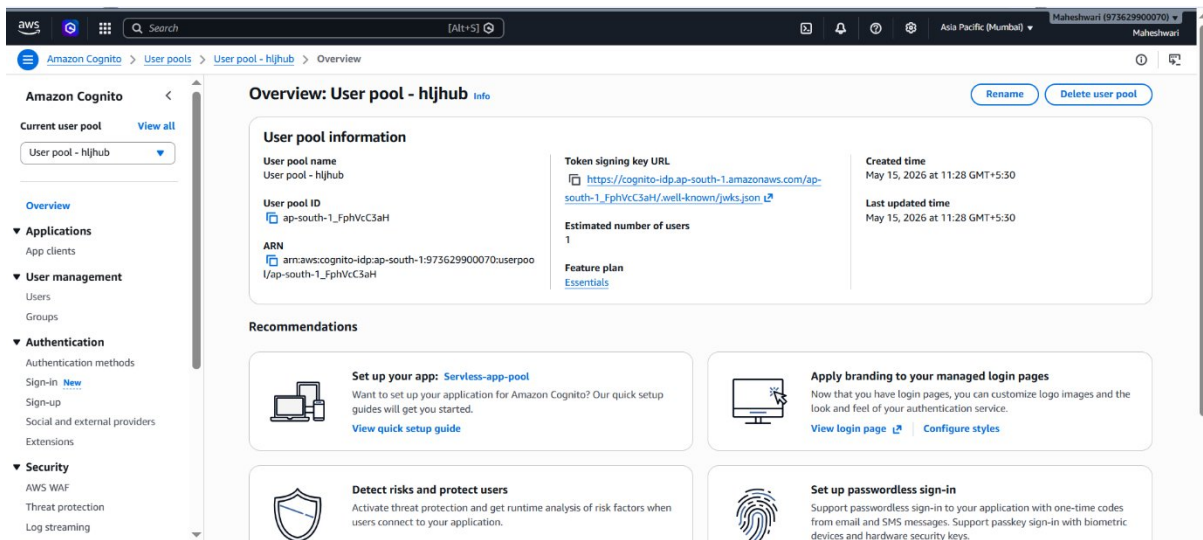
Step 3: Create Cognito User Pool

- Go to Cognito → Create User Pool.
- Configure sign-in options:
- Set password policy.
- Create App Client.
- Note the following:

* User Pool ID: ap-south-1_FphVcC3aH

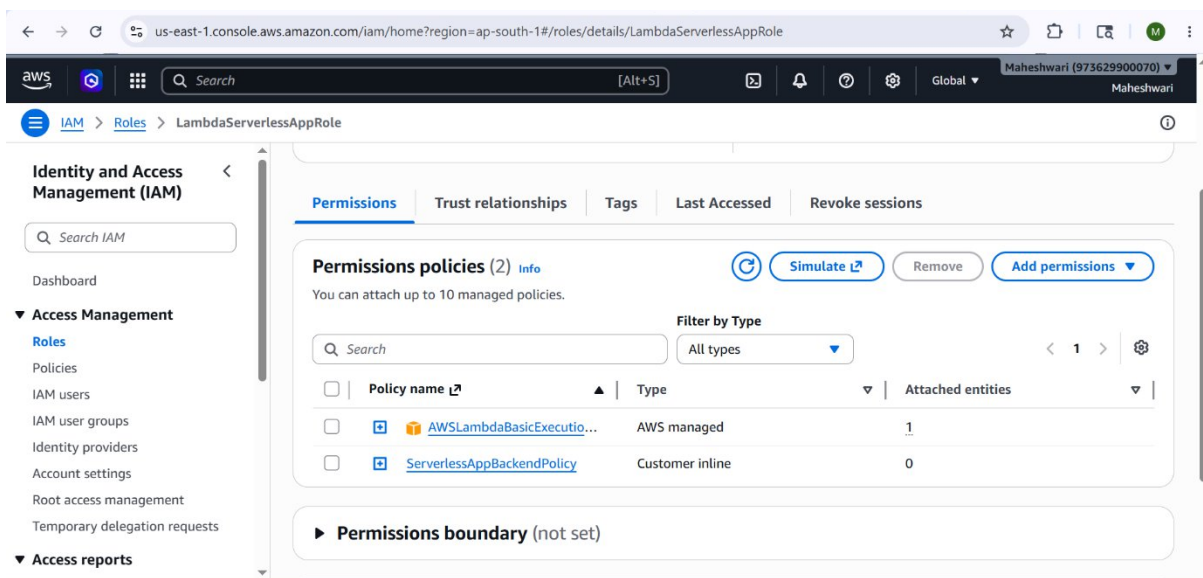
* App Client ID: 59jpdg0qem8f30pupiocrccc22

- Create User Pool.



Step 4: Create IAM Role for Lambda

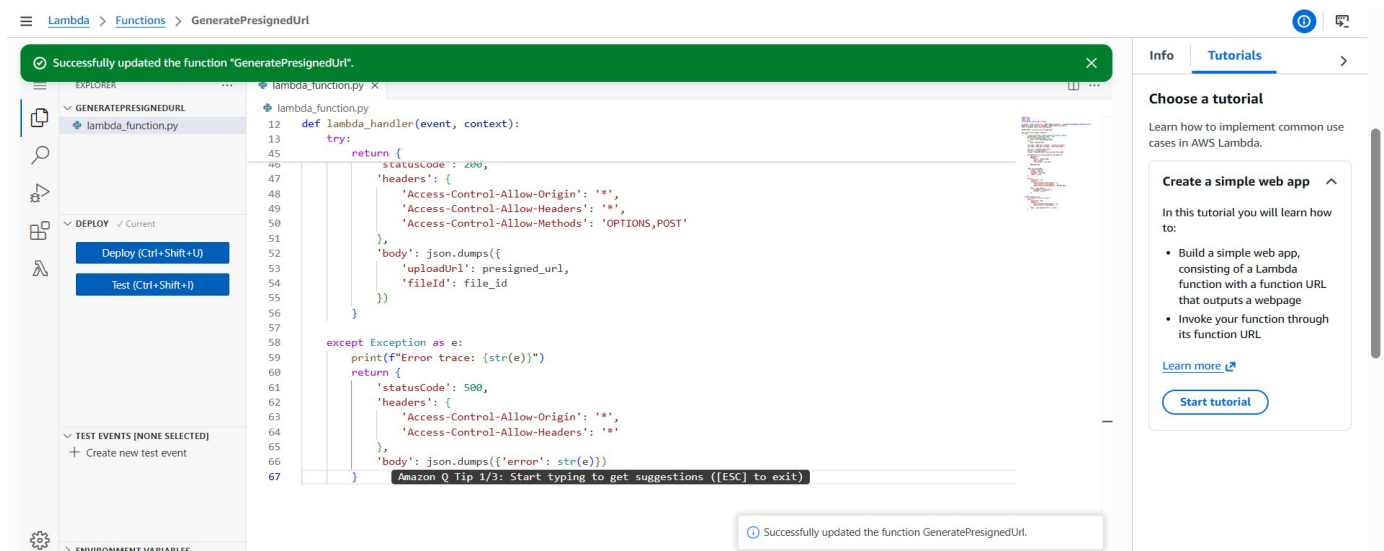
- Go to IAM → Roles → Create Role.
 - Trusted Entity:
 - Attach Policies:
- * AWSLambdaBasicExecutionRole
 - * AmazonS3FullAccess
 - * AmazonDynamoDBFullAccess
- Role Name: LambdaServerlessAppRole
 - Create Role.



Step 5: Create Lambda Function

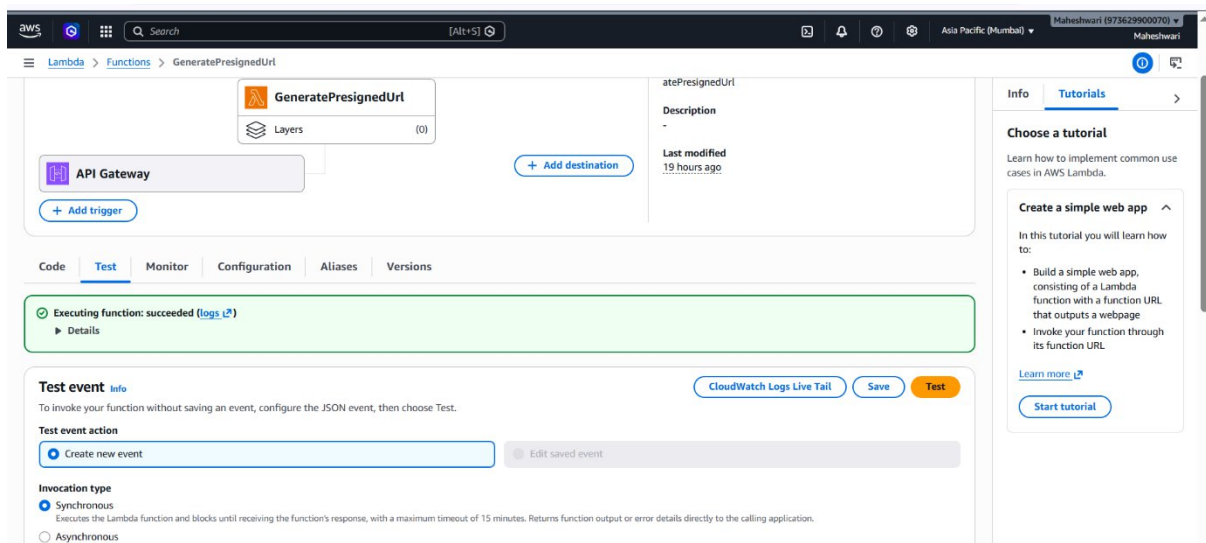
- Go to Lambda → Create Function.
- Function Name: GeneratePresignedUrl
- Runtime: Python 3.12
- Select Existing Role: LambdaServerlessAppRole

Step 6: Add Lambda Function Code



Step 7: Test Lambda Function

- In Lambda Console → Click Test.
- Create Test Event.
- Event JSON:
- Click Test.
- Verify:



Step 8: Create API Gateway

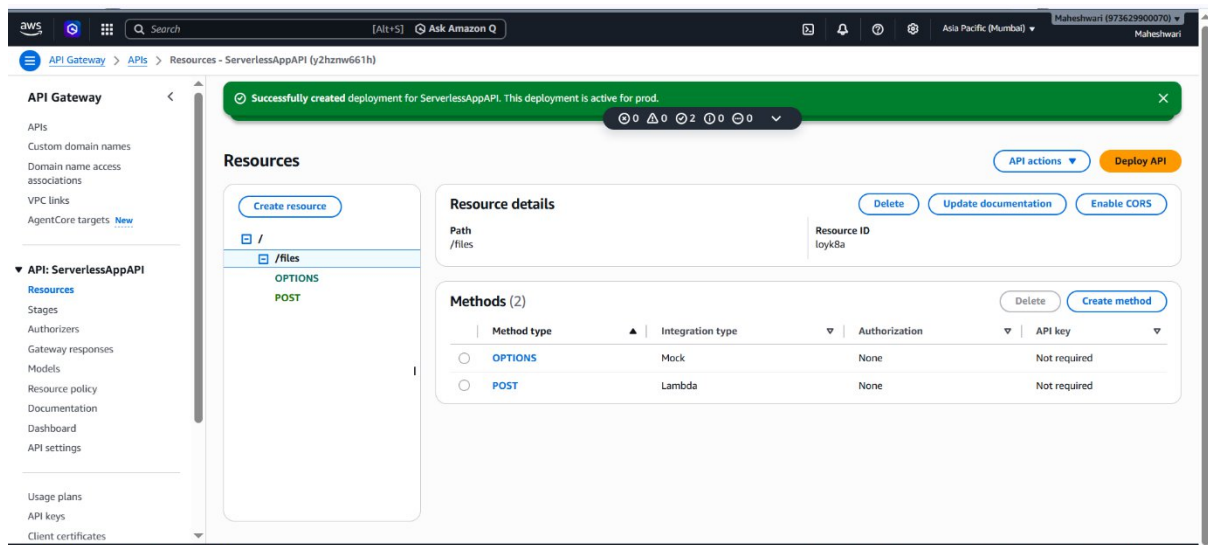
- Open Amazon API Gateway
- Create REST API
- Create resource:
- /files
- Create POST method
- Integrate with Lambda function
- Enable:
- Lambda Proxy Integration
- Deploy API

Step 9 — Configure API with Cognito

- In API Gateway:
 - Authorizers
 - Create Cognito Authorizer
- Select your User Pool
- Attach authorizer to POST method
- Now only authenticated users can access the API.
- Save changes.

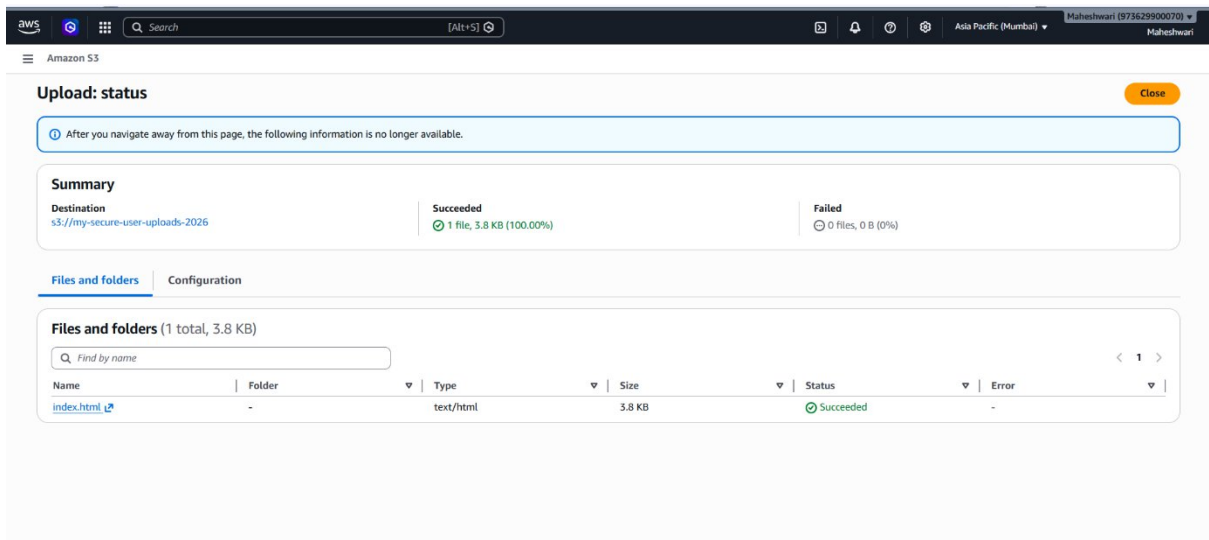
Step 10: Enable CORS

- In API Gateway → CORS.
- Enable:
 - * POST
 - * OPTIONS
- Save and Deploy API.



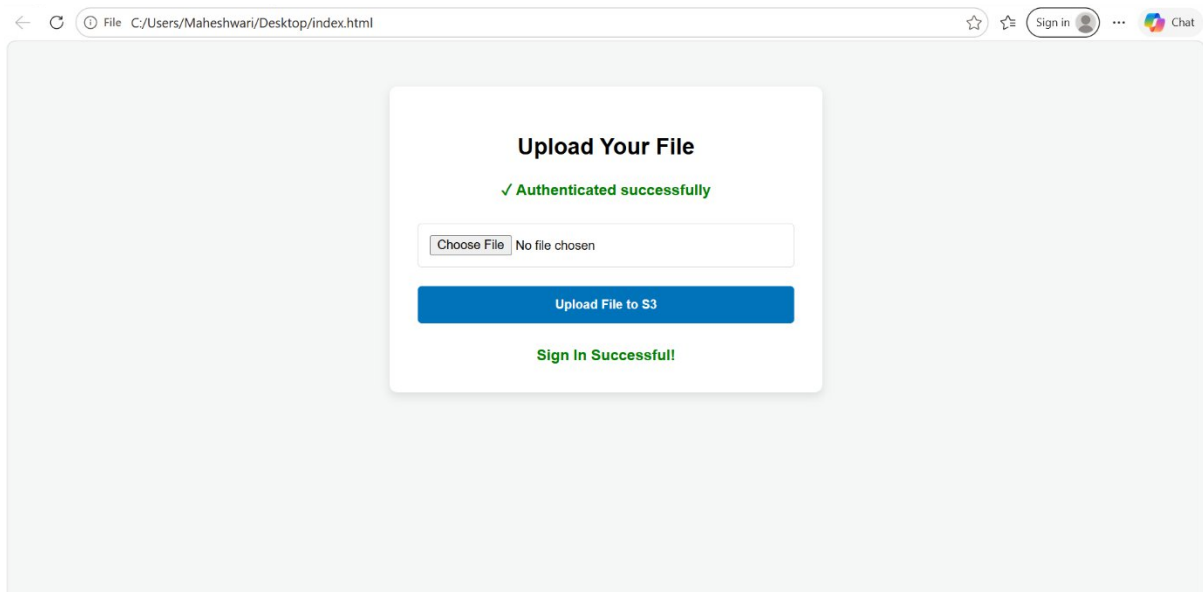
Step 11: Create Frontend Web Page

- Create an HTML page for login and file upload.
- Host frontend using: Amazon S3 Static Website Hosting

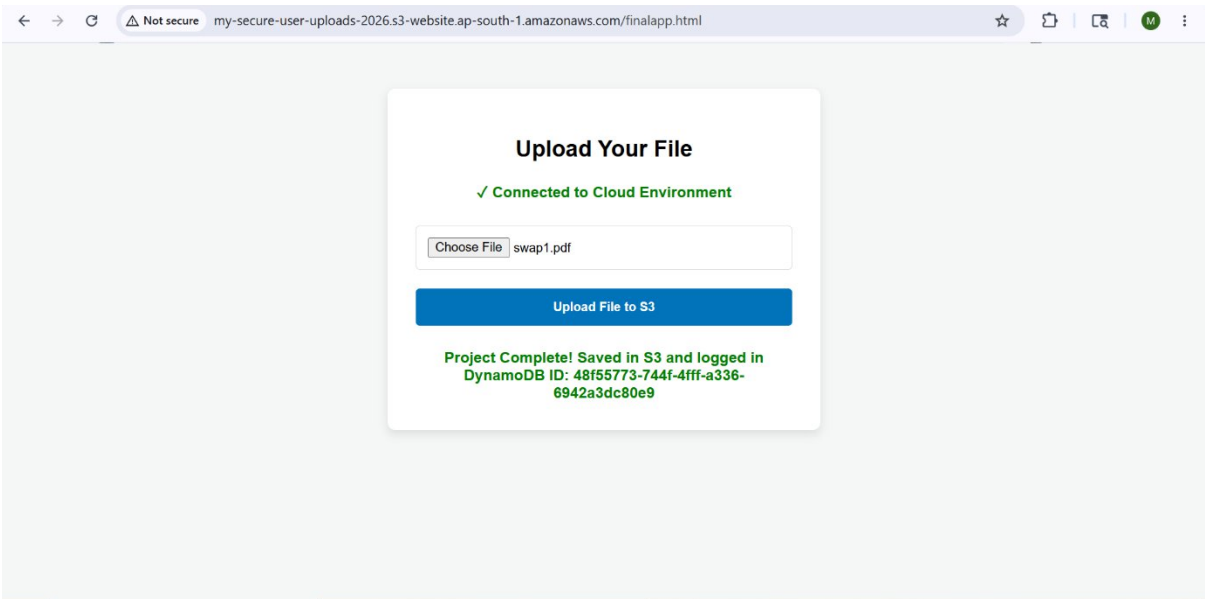


●Verify the Application

- Open the hosted web application.
- Register/Login using Cognito.
- API request processed successfully.

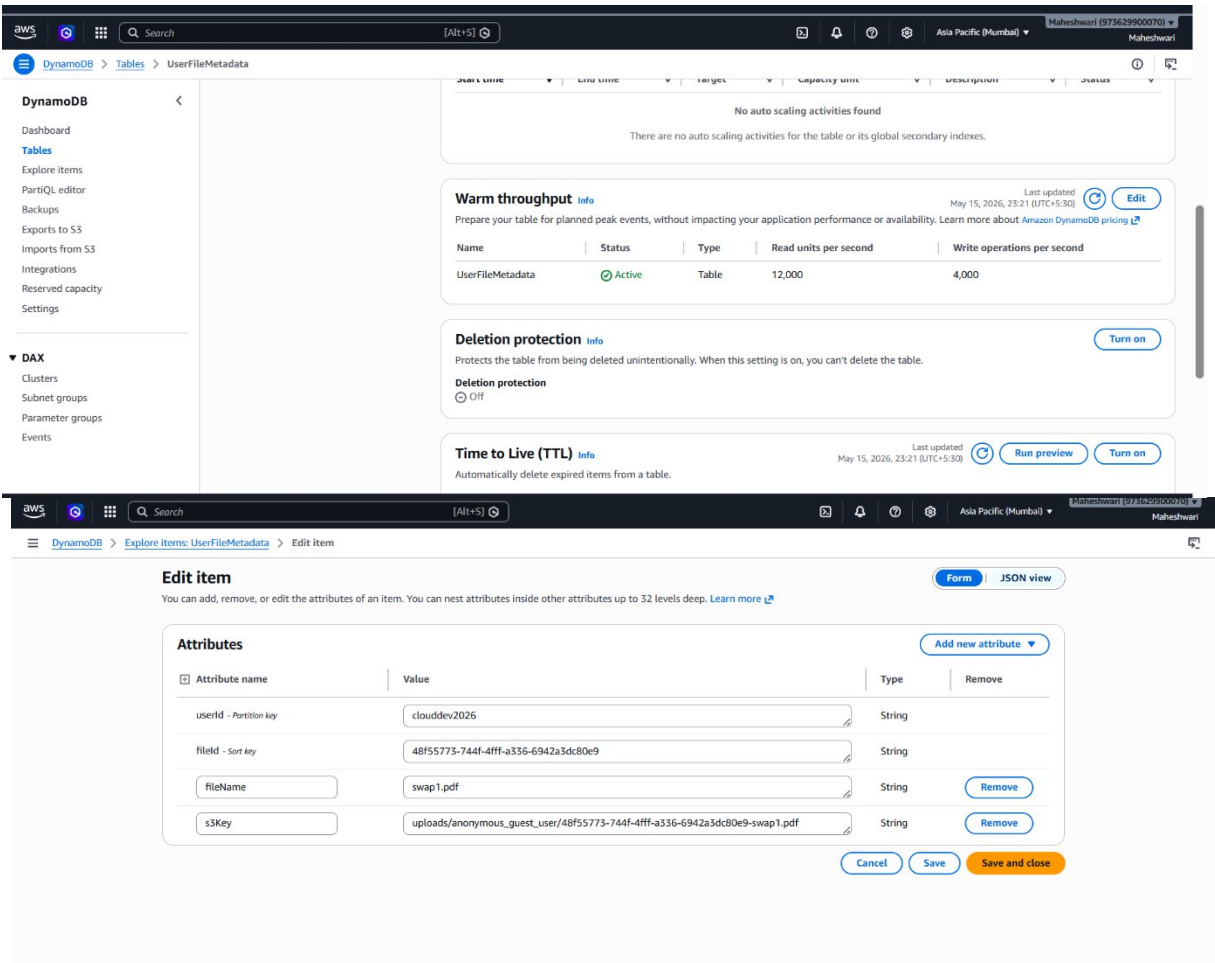


- Upload a file.



- Verify:

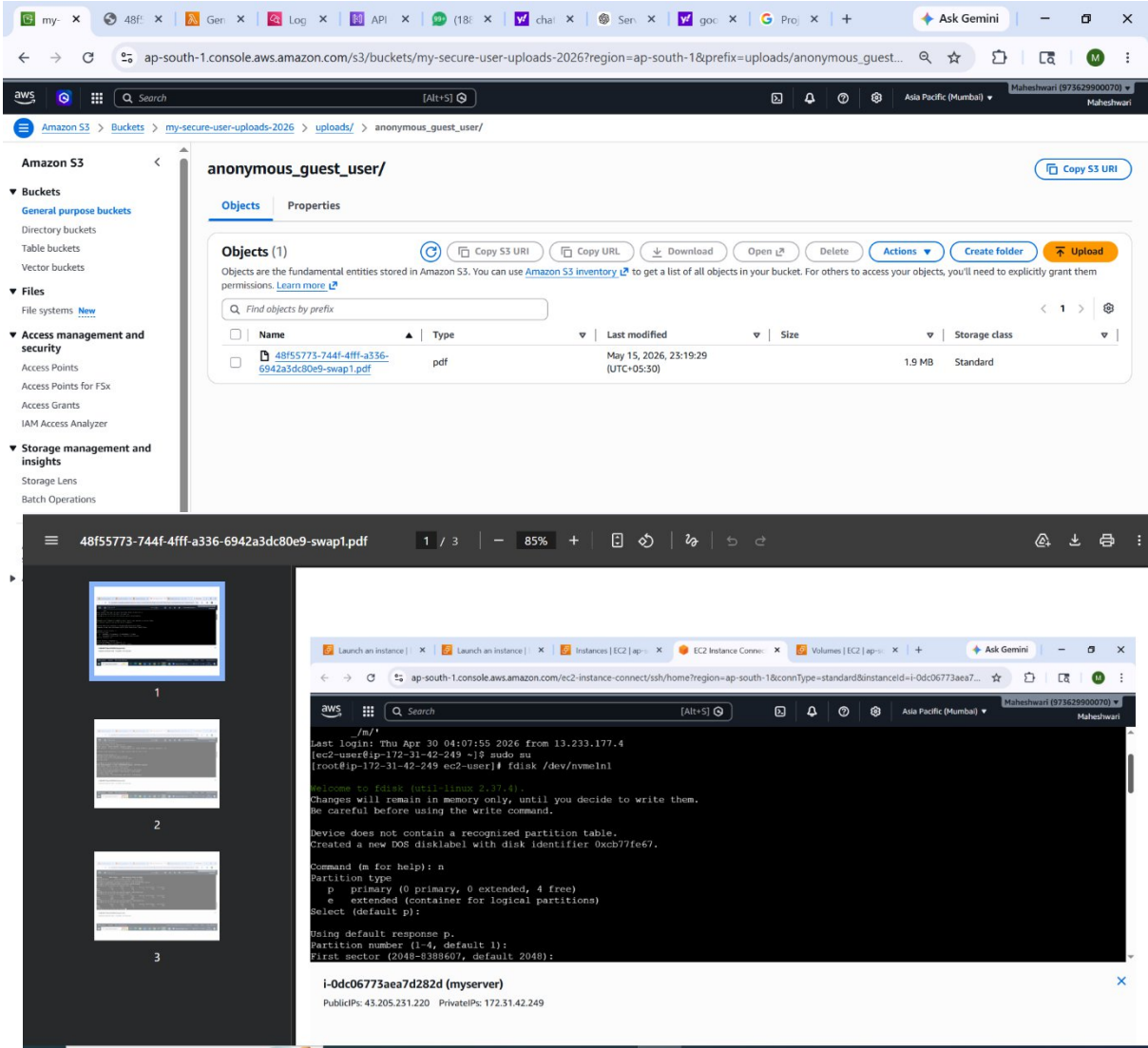
* Metadata stored in DynamoDB.



File exists in S3

File stored

in S3

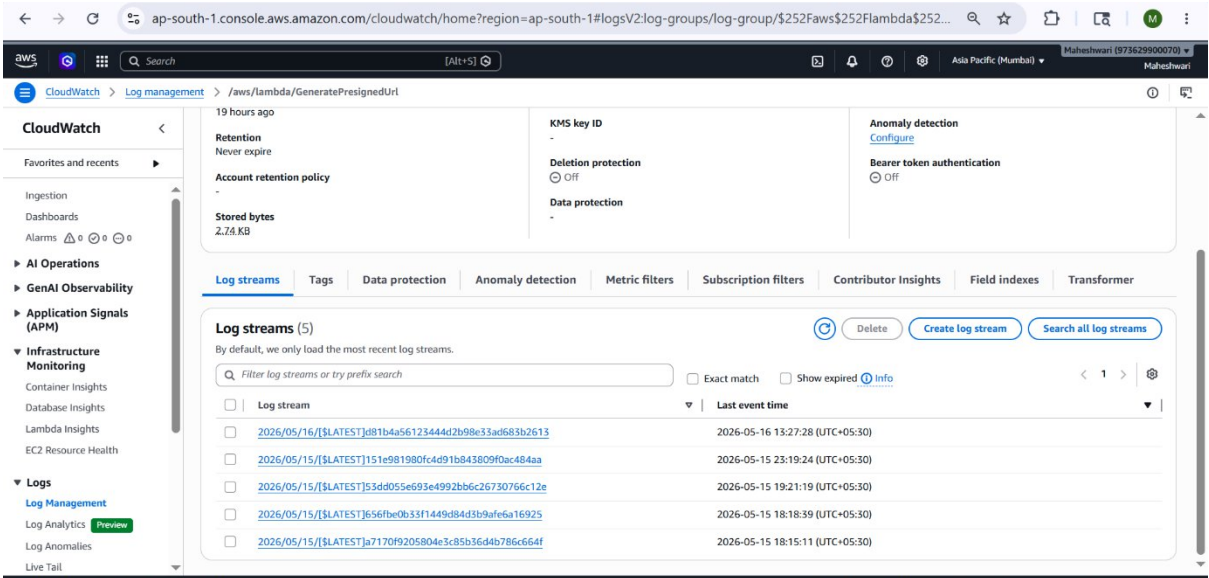


Monitoring

- Amazon CloudWatch

Monitor:

- Lambda logs



Troubleshooting:

- **Issue 1:** API Gateway Returns 403 Forbidden
- Cause: Authorization token missing or invalid.

Solution:

- * Verify Cognito Authorizer configuration.
- * Ensure valid JWT token is sent in request headers.

- **Issue 2:** Lambda Access Denied Error

Cause: Lambda IAM role lacks required permissions.

Solution:

Add permissions:

- * AmazonS3FullAccess
- * AmazonDynamoDBFullAccess

- **Issue 3:** CORS Error in Browser

Cause: CORS not enabled in API Gateway.

Solution:

- * Enable CORS for required methods.
- * Redeploy API.

- **Issue 4:** File Upload Failed

Cause: Incorrect bucket name or insufficient permissions.

Solution:

- * Verify S3 bucket name in Lambda code.
- * Check IAM permissions.

- **Issue 5:** Error 500 API Gate Way connection refused

Cause: Lambda crashed or returned invalid response.

Solution: Check Lambda Logs

Go to:

- * Amazon CloudWatch
- * Log Groups
- * /aws/lambda/YourFunctionName

- **Issue 5:** Error 404

The requested resource or endpoint does not exist.

Solution

Verify:

- API ID
- Region
- Stage name
- Resource path

Conclusion:

The Serverless Web Application provides a scalable, secure, and fully managed cloud-native architecture using AWS serverless services. By integrating Lambda, API Gateway, S3, DynamoDB, and Cognito, the application enables secure user authentication, file uploads, and metadata management without managing servers.

The solution is highly scalable, cost-efficient, and suitable for modern web applications. It reduces operational overhead while leveraging AWS managed services for security, availability, and automatic scaling.